TinyIDS: training run 1 (full dataset)

What the training code does, the first results, and what is good and what is not

Run date: 06 November 2026, on the Windows laptop (16 CPU cores, 15 GB RAM). The run took 32 minutes in total. All 12 models, the SHAP ranking and the RQ1 experiment are finished; all numbers below are final.

Contents

1 Files **2**

2 What the code does, step by step **2**

3 How the imbalance is handled **3**

4 Results (test set, 1,176,851 rows) **4**

5 Confusion matrices **6**

6 Against the proposal's targets **6**

7 What is good **7**

8 What is not good (yet), and the plan **7**

9 SHAP feature ranking (first part of RQ1) **8**

10 RQ1 result: how few features are enough? **9**

11 Next steps **9**

1 Files

File / folder	What it is
`train_tinyids.py`	The training script (run: `python train_tinyids.py`).
`train_tinyids.ipynb`	The same code as a Jupyter notebook, with an explanation before every step. Generated from the `.py`, so both are always identical.
`training_full.log`	Everything the script printed during this run, with timestamps.
`cache/`	Fast **Parquet** copies of the three CSV files (made on the first run; later runs load in seconds).
`results/`	`results_summary.csv` (main table), `report_<task>_<model>.csv` (per-class scores), `cm_<task>_<model>.png` (confusion matrices).
`models/`	The trained models: `.joblib` (Random Forest, LightGBM), `.txt` (LightGBM, portable), `.pt` (PyTorch MLP / 1D-CNN).

2 What the code does, step by step

1. **Load the data**: the **full** Kaggle split of CICIoT2023 (Canadian Institute for Cybersecurity IoT dataset 2023): train 5,491,971 rows, validation 1,176,851, test 1,176,851; 46 features + 1 label. Numbers stored as `float32`; the few infinite values replaced by 0.
2. **Make three label sets**: **binary** (Benign / Attack), **8-class** (family) and **34-class** (exact attack).
3. **Handle the class imbalance** with **square-root class weights** (section 3).
4. **Train four models** for each label set: **RF** (Random Forest), **LightGBM** (Light Gradient Boosting Machine), **MLP** (Multi-Layer Perceptron) and **1D-CNN** (one-dimensional Convolutional Neural Network). 12 models in total.
5. **Use the validation set** to stop training at the right moment (LightGBM "early stopping", and picking the best epoch for MLP / CNN).
6. **Score each model on the test set**, which the models never saw during training.
7. **Save** every model, table and plot.
8. **RQ1** (Research Question 1): rank the features with **SHAP** (SHapley Additive exPlanations), retrain with the top 5 / 10 / 15 / 20 features, compare with all 46.

Model settings used

Model	Settings
Random Forest	100 trees, max depth 20, each tree trained on 500,000 rows drawn from the full training set
LightGBM	up to 400 boosting rounds, learning rate 0.1, 63 leaves per tree, early stopping after 20 rounds without improvement
MLP	$46 \to 128 \to 64 \to$ classes, 3 epochs, Adam optimiser, batch 4096
1D-CNN	2 convolution layers (32 filters, width 3) $\to 64 \to$ classes, 3 epochs

For MLP and 1D-CNN the features are first transformed with $\text{sign}(x) \cdot \log(1 + |x|)$ and standardised using the mean and standard deviation of the **training set only** (as the proposal requires), because values range from 0–1 (flags) to millions (`Rate`).

3 How the imbalance is handled

DDoS is 73% of all rows; BruteForce is 0.03%. A model trained naively learns "always DDoS" and still scores 73% accuracy.

- **Undersampling** (throwing away most DDoS rows) was **not** used, because the full dataset was wanted.
- **Class weights** were used: every row is kept, but mistakes on rare classes cost more during training.
- The fully "balanced" weight $N/(K \cdot n_c)$ gives the rarest classes a weight of about 1,000 and makes training unstable. We use its **square root** ("tempered" weighting): rare classes count about 30× more than DDoS.
- The main score is **macro-F1**, where every class counts equally, so a model cannot look good by only getting DDoS right.

4 Results (test set, 1,176,851 rows)

Task	Model	Accuracy	Macro-F1	Benign FPR	Size (KB)	Latency (ms/row)	Train (s)
binary	RF	0.9961	0.9607	0.0027	6,860	7.43	44
binary	**LightGBM**	**0.9969**	**0.9677**	0.0098	2,668	1.15	43
binary	MLP	0.9918	0.9232	0.0084	61	0.14	70
binary	1D-CNN	0.9914	0.9200	0.0138	83	0.31	234
8-class	**RF**	**0.9955**	**0.8638**	0.0427	33,612	7.87	48
8-class	LightGBM	0.9954	0.8161	0.0445	10,642	1.76	176
8-class	MLP	0.8995	0.6721	0.0748	62	0.15	74
8-class	1D-CNN	0.8816	0.6366	0.1420	84	0.67	259
34-class	**RF**	**0.9929**	**0.7961**	0.0392	46,152	9.24	78
34-class	LightGBM	0.9895	0.7128	0.1106	4,269	1.49	158
34-class	MLP	0.8598	0.5929	0.1211	70	0.14	76
34-class	1D-CNN	0.8523	0.5658	0.1284	91	0.39	242

- **Benign FPR** (False Positive Rate) = share of normal (benign) traffic wrongly flagged as an attack. Lower is better.
- **Size** = saved model file; **Latency** = time to classify one single row on the laptop; **Train** = training time.
- Batch throughput (many rows at once) was 35,000 to 3,000,000 rows/s for all models, far above the target of 500 flows/s.

Per-class results: binary, LightGBM

Class	Precision	Recall	F1	Test rows
Attack	0.9998	0.9970	**0.9984**	1,149,142
Benign	0.8891	0.9902	0.9369	27,709

Per-class results: 8-class, Random Forest (best 8-class model)

Class	Precision	Recall	F1	Test rows
DDoS	0.9999	0.9996	0.9997	855,981
DoS	0.9996	0.9995	0.9996	204,245
Mirai	0.9996	0.9993	0.9995	66,749
Benign	0.9295	0.9573	0.9432	27,709
Spoofing	0.8924	0.8571	0.8743	12,410
Recon	0.8169	0.8467	0.8315	8,812
BruteForce	0.9250	0.5799	0.7129	319
Web	0.7078	0.4489	**0.5494**	626

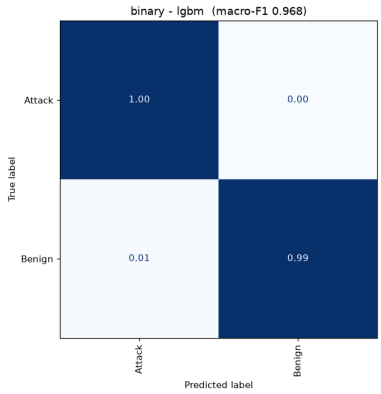
Weakest classes: 34-class, Random Forest

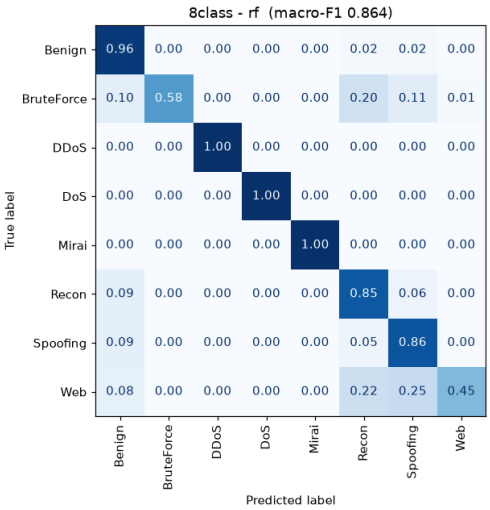
Attack	Recall	F1	Test rows
Uploading_Attack	0.061	0.108	33
Backdoor_Malware	0.191	0.293	89
Recon-PingSweep	0.208	0.293	53
XSS	0.214	0.331	103
SqlInjection	0.230	0.337	148
BrowserHijacking	0.358	0.482	134
CommandInjection	0.403	0.530	119
DictionaryBruteForce	0.505	0.640	319

All the weak ones are **tiny classes** (33 to 319 test rows). The flood attacks are almost perfect.

5 Confusion matrices

Rows = true class, columns = predicted class, each row sums to 1. A perfect model has 1.00 on the diagonal.





Left: binary LightGBM. Right: 8-class Random Forest. Only 45% of Web attacks are recognised: 25% are mistaken for Spoofing, 22% for Recon and 8% for Benign. BruteForce: 58% correct, 20% mistaken for Recon, 11% for Spoofing, 10% for Benign. The big families (DDoS, DoS, Mirai) are at 1.00. All other confusion matrices (including 34-class) are in `model/results/`.

6 Against the proposal's targets

Target	Required	Run 1 (best model)	Met?
Binary F1	≥ 0.98	Attack-class F1 **0.998**; macro-F1 (Attack + Benign averaged) 0.968	**OK** / partly
Model size	< 1 MB	Trees 2.7–46 MB; MLP / CNN 61–84 KB	**NO** for trees
Latency per flow	< 5 ms	LightGBM 1.2–1.8 ms, MLP/CNN $<$ 0.7 ms, RF 7–9 ms (laptop)	**OK** except RF
Throughput	≥ 500 flows/s	35,000+ rows/s in batches	**OK**
8-class within 2% (RQ1)	macro-F1 within 0.02 of all 46	best: 15 features, 0.027 below	**NO** (close)
SHAP time per alert	< 50 ms	not measured yet	—

Note: the Pi 4 is roughly 3–5× slower than this laptop, so latency must be measured again on the Pi.

7 What is good

- **Binary detection works very well**: 99.7% accuracy, attack-class F1 0.998, and RF flags only 0.27% of normal traffic as attacks.
- **The attacks of the live demo are detected almost perfectly**: DDoS, DoS and Mirai floods have F1 $\approx$ 0.9996. SYN / UDP floods (hping3) fall in these families.
- **Using the full dataset was fast**: the whole run took about 30 minutes (training is done once; afterwards the saved models are just loaded).
- **Tree models beat the neural networks**, as expected for table data, and they are what TreeSHAP explains best.
- **MLP and 1D-CNN are already tiny** (61–84 KB) and very fast, so they meet the size target without any compression.
- The pipeline is complete and reproducible: one command trains, scores and saves everything.

8 What is not good (yet), and the plan

Problem	Why / plan
Rare classes are weak: Web (F1 0.55), BruteForce (0.71), Recon (0.83); in 34-class some attacks have recall below 25%.	Only 33–626 test rows, and these attacks look like normal web traffic in packet statistics. Plan: try stronger class weights for 8-class; report honestly. None of them is in the live demo except MQTT brute force.
Tree models are too big (2.7–46 MB vs target $<$ 1 MB).	Expected before compression (that is RQ2). Plan: fewer and shallower trees, top-k features from RQ1.
Random Forest single-row latency 7–9 ms ($>$ 5 ms).	100 deep trees. Compression (above) will cut this; LightGBM is already 1.2–1.8 ms.
LightGBM is worse than RF on 8 and 34 classes, and its 34-class benign false-alarm rate is high (11%).	Unusual; probably the learning rate (0.1) is too high for many classes with weights. Plan: retrain with a lower learning rate.
MLP / 1D-CNN are clearly weaker (8-class macro-F1 0.64–0.67) and have higher false-alarm rates.	Only 3 epochs and very small networks. Acceptable: they are mainly the comparison point for RQ2.
Binary macro-F1 0.968 is below 0.98 if the target means macro-F1.	The Benign class (2.4% of rows) has precision 0.89: some attack rows are classified as benign. Attack-class F1 is 0.998. Clarify which F1 the target means.
Likely "shortcut" feature `IAT` (Inter-Arrival Time).	**Confirmed on the full data:** SHAP importance of `IAT` is 1.31, about 5$\times$ the next feature (`Protocol Type`, 0.27). Its values in this dataset look like capture timestamps rather than traffic behaviour, so the model may partly recognise *when/where* a capture was made instead of *what* the attack does; a live IDS cannot reproduce that. Plan: retrain without `IAT` and compare.
Memory reached 97% of 15 GB during training.	The script holds several copies of the data at once. Plan: free memory between steps before the next run.

9 SHAP feature ranking (first part of RQ1)

Mean absolute SHAP value of each feature for the 8-class LightGBM model (3,000 validation rows). Higher = more influence on the decision.

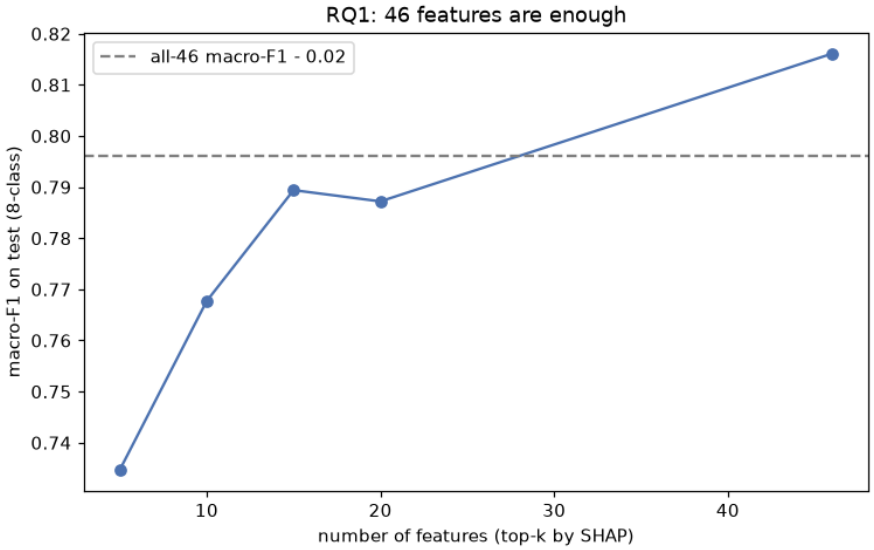
#	Feature	SHAP	#	Feature	SHAP
1	IAT	**1.313**	11	Tot sum	0.111
2	Protocol Type	0.267	12	urg_count	0.099
3	Min	0.238	13	rst_count	0.093
4	Max	0.197	14	ack_count	0.090
5	Header_Length	0.194	15	Variance	0.088
6	Tot size	0.164	16	AVG	0.088
7	Rate	0.137	17	syn_count	0.081
8	Number	0.136	18	syn_flag_number	0.079
9	Magnitue	0.123	19	fin_count	0.063
10	flow_duration	0.118	20	ICMP	0.054

After IAT, the ranking makes sense for a network IDS: protocol, packet sizes, rate and TCP flag counts.

10 RQ1 result: how few features are enough?

LightGBM (8-class) retrained with only the top-k features of the SHAP ranking, scored on the full test set:

Features (k)	Macro-F1	Gap to all 46	Within 0.02?
5	0.7347	−0.081	no
10	0.7677	−0.048	no
15	**0.7894**	−0.027	no (closest)
20	0.7872	−0.029	no
46 (all)	0.8161	—	reference



Answer from run 1: none of the reduced sets stayed within 0.02 of the full 46-feature model. The best was **15 features** (macro-F1 0.789, 0.027 below), and going from 15 to 20 brought no further gain. Top-15 features: `IAT`, `Protocol Type`, `Min`, `Max`, `Header_Length`, `Tot size`, `Rate`, `Number`, `Magnitue`, `flow_duration`, `Tot sum`, `urg_count`, `rst_count`, `ack_count`, `Variance`.

Caveats:

- The reference model (8-class LightGBM, 0.816) is itself weaker than Random Forest (0.864) in this run; with a better-tuned model the picture may change.
- The gap between 15 and 46 features comes almost entirely from the rare classes (Web, BruteForce); the large classes are near-perfect with few features.
- `IAT` is first in every top-k set; RQ1 must be repeated without it (see the `IAT` problem above).
- Steps of 5 are coarse: also testing 25 and 30 features would show where the curve reaches the 0.02 line.

11 Next steps

1. Retrain without `IAT` and repeat RQ1 (also with 25 and 30 features, and with Random Forest as the reference).
2. Retrain LightGBM with a lower learning rate; reduce memory use.

3. Compression for RQ2: smaller trees, export to ONNX (Open Neural Network Exchange) / TFLite (TensorFlow Lite), measure on the Pi.